

London Tech Week 2026

RAG-Powered Chatbot with Hybrid Q/A Retrieval

Project Report

Chantal Sandal

Building Agentic AI Applications for Beginners — Bootcamp Contest #1

Course/Batch: AGAI-03

Repository: github.com/Csandal17/rag-chatbot-ltw

May 2026

Contents

1. Introduction and Website Choice
2. Data Collection (Scraping)
3. Synthetic Q/A Generation
4. Vector Database
5. Hybrid Retrieval (Core Logic)
6. User Interface (Streamlit)
7. Challenges, Debugging, and Learnings
8. Limitations and Future Improvements
9. Architecture Summary
10. Conclusion

1. Introduction and Website Choice

This report explains how I designed and built a Retrieval-Augmented Generation (RAG) chatbot for the Building Agentic AI Applications for Beginners bootcamp. The chatbot helps answer questions about London Tech Week 2026. This five-day technology festival takes place from 8 to 12 June 2026 at Olympia London.

I chose London Tech Week as the focus because I will be attending the event in person this year. It is one of the largest free technology events in the UK. The event offers great networking and learning opportunities for students and people changing careers. Building an assistant for an event I will attend makes the project feel real and important. It also motivated me to ensure the chatbot was accurate, since any mistakes could affect my own experience at the event.

The website (londontechweek.com) is also well-suited to the assignment. It is content-rich, with hundreds of sessions and speaker pages, and it is publicly available. The goal of the project was to build a chatbot that could act as a knowledgeable assistant for that content and answer attendees' questions about sessions, speakers, the schedule, and practical event information.

Technology stack

The project was built entirely in Python 3.12. The main libraries and tools were:

- Scraping: requests, BeautifulSoup4, lxml
- LLM (Q/A generation and Tier 2 answers): the Anthropic Claude API, model claude-sonnet-4-6
- Vector database: ChromaDB, using its default embedding model, sentence-transformers all-MiniLM-L6-v2 (runs locally, no API cost)
- User interface: Streamlit
- Supporting tools: pandas, python-dotenv, and Git/GitHub for version control

2. Data Collection (Scraping)

Data collection was handled by `scraper.py`, which used Python's requests library to fetch pages and BeautifulSoup (with the lxml parser) to extract content. Rather than guessing at URLs or crawling blindly, I read the site's XML sitemaps: the sitemap index points to sub-sitemaps for speakers and for the agenda, which, between them, list every page worth scraping. This is a cleaner and more respectful approach than following links at random, because it uses the structure the site itself publishes.

What was scraped

From the sitemaps I scraped:

- 50 speaker pages — capturing each speaker's name, position, and company.
- Approximately 300 session pages — capturing each session's title, time, stage, theme, and description. After cleaning (see below), this came to 299 sessions.

This comfortably exceeds the assignment's minimum of 10–20 pages. The raw results were saved as structured JSON files (speakers.json and sessions.json) in the data/raw/ folder. To scrape politely and avoid overloading the server, the scraper waits 0.5 seconds between requests.

A note on the site being server-rendered

Before writing the scraper, I inspected the page source to check how the site delivers its content. London Tech Week's pages are server-rendered with clean, consistent CSS class hooks, which means the full content is present in the initial HTML response. This was an important finding: it meant plain requests and BeautifulSoup were sufficient, and I did not need a heavier tool such as Selenium or a JavaScript renderer. Validating this up front saved a great deal of unnecessary complexity.

Scraping challenges

Two specific problems came up during scraping, both worth recording:

1. **Sitemap parsing.** BeautifulSoup needed the "lxml-xml" parser to read the XML sitemaps correctly. My first attempt threw a FeatureNotFound error because the lxml parser was not installed. Installing lxml and specifying the right parser resolved it. This was a useful reminder that XML and HTML are parsed differently.
2. **Junk data on the agenda landing page.** The /agenda landing page came through the sitemap and was initially scraped as if it were a session, but with a stage value of "Unknown" and no real session content. I filtered this out, which is why the session count dropped from 300 to 299.

A known limitation in the data

Sessions and speakers were scraped into separate files and are not linked to one another. As a result, a question such as “who is speaking at session X?” cannot always be resolved to a named person, because the connection only exists if the session description itself names the speaker. This is a genuine limitation of the current dataset, and linking speakers to sessions is listed as a future enhancement in Section 8.

3. Synthetic Q/A Generation

With the raw data collected, the next phase was to generate question-and-answer pairs from it. This is the dataset the chatbot retrieves against. I used the Anthropic Claude API (model claude-sonnet-4-6) to generate these pairs from the scraped content.

Generation strategy

I generated 150 Q/A pairs by sampling 50 items from the scraped data (35 sessions and 15 speakers) and asking Claude to produce 3 question-answer pairs for each. Several deliberate choices went into this process:

- **Reproducible sampling.** The 50 items were chosen with random.sample using a fixed seed (random.seed(42)), so the same sample is selected on every run. This makes the generation reproducible — an important property for debugging and reliably rerunning.

- **Grounded prompting.** The prompt instructed Claude to use only the supplied information for each item. This is the central discipline of a RAG system: it keeps the generated answers tied to the source data and reduces the risk of the model inventing details.
- **Structured JSON output.** Claude was prompted to return only a valid JSON array of {question, answer} objects. Because models sometimes wrap output in markdown code fences, the code defensively strips those fences before calling `json.loads`, so parsing does not break.
- **Robust error handling.** Each API call is wrapped in a try/except block, so a single problematic item is skipped rather than crashing the whole batch. On the full run, there were zero skips.
- **Cost-conscious model choice.** I chose Claude Sonnet rather than the more powerful Opus model. The task — turning structured facts into clear Q/A pairs — did not require maximum capability, so Sonnet was the more cost-efficient choice.

Curating and extending the dataset by hand

After generating the initial 150 pairs and testing the chatbot, I found a gap. The generated pairs covered sessions and speakers well, but they did not cover the practical, real-world questions that attendees actually ask, like “Is there a hackathon?”, “Can I bring my children?”, “Where do I collect my badge?”, and “Is there a quiet space?”

To address this, I added 22 further Q/A pairs by hand, bringing the dataset to 172 pairs in total. Crucially, I verified every one of these additions against the live London Tech Week website (the FAQ, hackathon, childcare, and SensoryCalm Quiet Room pages) before adding them, and I tagged each with a descriptive source label. During this process, I also noticed that an AI code-completion tool had suggested several plausible-looking Q/A rows whose facts were not actually on the site; I removed those and replaced them with verified content. This experience reinforced the core RAG principle in a very concrete way: the value of the system depends entirely on the answers being grounded in real, checkable sources rather than confident-sounding invention.

The final dataset was saved to `data/processed/qa_pairs.csv` using Python's `csv.DictWriter`, which automatically quotes any values containing commas. The columns are source, question, and answer.

4. Vector Database

The Q/A pairs are stored in a vector database so they can be searched by meaning rather than by exact keywords. I used ChromaDB for this, built by `vector_store.py`.

Embeddings and the embedding model

An embedding is a way of turning a piece of text into a list of numbers (a vector) that captures its meaning. Texts with similar meaning produce vectors that sit close together; texts with different meanings sit far apart. This is what lets the chatbot match a user's question to a stored question, even when the wording is completely different.

The embeddings are produced by Chroma's default model, the sentence-transformers model `all-MiniLM-L6-v2`. This is used implicitly: because `collection.add(documents=...)` is called without

specifying an embedding function, Chroma falls back to its built-in default. A useful property of this model is that it runs locally on the machine, so generating embeddings costs nothing in API fees. I confirmed which model was in use from the cache path printed on the first run (.cache/chroma/onnx_models/all-MiniLM-L6-v2).

How the data is stored

- Each question is stored as a document (the thing that gets embedded and searched).
- The corresponding answer and source are stored as metadata attached to that document, so they can be retrieved alongside the match.
- The database is persisted to disk using PersistentClient(path="chroma_db"), so it does not need to be rebuilt every time. The chroma_db/ folder is gitignored because it can always be regenerated from the CSV.
- The build script uses a delete-then-recreate pattern: if the collection already exists, it is deleted first, then created fresh. This makes the script safe to re-run without piling up duplicate entries.

Distances have structure

A helpful insight from testing is that the distance scores returned by the vector search are meaningful, not arbitrary. For the test query “what time is the de-extinction session?”, the results showed a clear pattern: a distance of about 0.409 for a match with the same topic and the same intent, around 0.631 for the same topic but a different intent, and around 0.737 for a different topic with a similar intent. Understanding this structure is what made it possible to choose a sensible threshold for the hybrid logic described next.

5. Hybrid Retrieval (Core Logic)

The heart of the project is the hybrid retrieval logic in retriever.py. When a question arrives, the system embeds it, searches the vector database for the closest stored questions, and then makes a decision based on how close the best match is. This produces a two-tier system.

Tier	Condition	What happens
Tier 1 — Direct hit	Closest distance < 0.5	The stored answer is returned immediately. Fast, and uses no LLM API call.
Tier 2 — Fallback	Closest distance ≥ 0.5	The top 3 related pairs are retrieved as context and handed to Claude, which synthesises a grounded answer (RAG).

The threshold is set as a named constant (DISTANCE_THRESHOLD = 0.5), which makes it easy to find and tune. The value of 0.5 was chosen from the testing described above: good matches tend to score around 0.4, while genuine misses score well above 1.3, so a threshold of 0.5 cleanly separates a confident match from a question the dataset does not directly answer.

How it behaves in testing

- “What time is the de-extinction session?” → distance 0.409 → **Tier 1 direct hit**.
- “pizza” → distance 1.512 → **Tier 2** (correctly recognised as having no confident match).
- “failure or resilience” → distance 1.343 → **Tier 2**, where Claude synthesised an answer and honestly flagged that the information was incomplete.

Honest fallback behaviour

A point I am particularly pleased with is how the Tier 2 fallback behaves when it does not know something. Because the prompt instructs Claude to use only the provided context, the model admits uncertainty rather than inventing an answer. For a factual event assistant, this is exactly the right behaviour — it is far better for the bot to say it does not have that information than to confidently state a wrong session time.

A tuning insight

Testing also revealed an honest limitation in the Tier 2 path. The quality of a synthesised answer depends on the right context being among the top 3 retrieved pairs. In one case (“Failure Isn’t Fatal”), a relevant pair existed in the data but was not retrieved into the top 3 for that particular query, so it could not inform the answer. Retrieving more results, or further tuning the threshold against a set of evaluation questions, would improve this. It is noted as a future improvement.

6. User Interface (Streamlit)

The chatbot is wrapped in a Streamlit application (app.py) that provides a clean, branded chat interface. The interface was styled in London Tech Week's colours — a deep blue header banner on a light background — to make it feel purpose-built for the event.

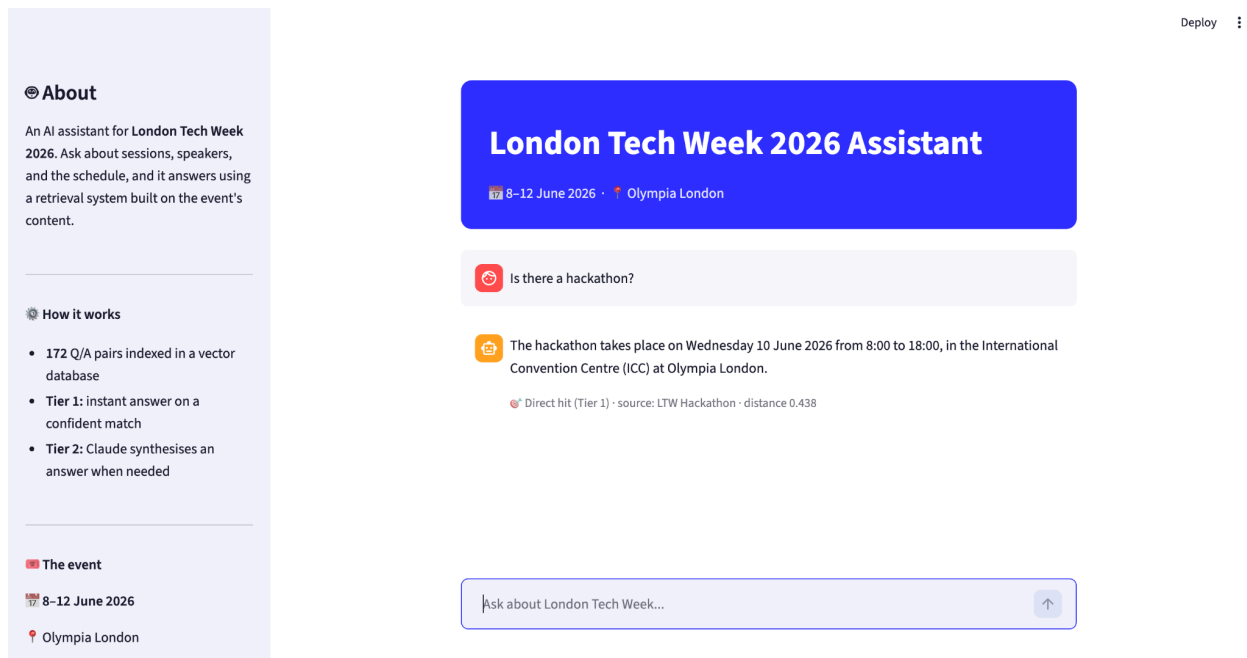
The application includes the following features:

- **A conversational chat interface** built with Streamlit's chat components. Questions and answers are displayed as chat bubbles, and the conversation history stays on screen.
- **Chat memory.** The conversation is stored in Streamlit's session state, so previous exchanges persist as the user asks follow-up questions, rather than each question being answered in isolation.
- **Source citation.** Under each answer, a small caption shows which tier answered (Tier 1 direct hit or Tier 2 synthesis), the source label of the matched Q/A pair, and the distance score — so the user can see where the answer came from and how confident the match was.
- **A “Clear chat” button** in the sidebar that resets the conversation.
- **An informative sidebar** with an “About” description, a “How it works” summary (number of Q/A pairs indexed and an explanation of the two tiers), and key event details with a link to the official website.

For performance, the retriever (the “brain”) is loaded once and cached with Streamlit's `@st.cache_resource` decorator, so the vector database is opened a single time rather than on every interaction.

Screenshots

Screenshots of the working chatbot are included in the repository README, showing: a Tier 1 direct hit (for example, “Is there a hackathon?” returning the stored answer with a “Direct hit (Tier 1)” source citation), and a Tier 2 synthesised answer (for example, “Can I bring my children?”, where Claude combines several childcare-related pairs into one organised reply). The screenshots also show the branded header, the sidebar with project information and statistics, and the source-citation captions beneath each answer.



Deploy ⋮

About

An AI assistant for **London Tech Week 2026**. Ask about sessions, speakers, and the schedule, and it answers using a retrieval system built on the event's content.

How it works

- 172 Q/A pairs indexed in a vector database
- **Tier 1:** instant answer on a confident match
- **Tier 2:** Claude synthesises an answer when needed

The event

8–12 June 2026

Olympia London

How it works

- 172 Q/A pairs indexed in a vector database
- **Tier 1:** instant answer on a confident match
- **Tier 2:** Claude synthesises an answer when needed

The event

8–12 June 2026

Olympia London

londontechweek.com

Clear chat

London Tech Week 2026 Assistant

8–12 June 2026 · Olympia London

Your question:

Can I bring my children?

Bringing Children to London Tech Week

Yes, you can bring your children! Here are the key details:

- **Infants under 2** may accompany a parent throughout the entire venue
- **Children aged 2–17** are welcome at the **dedicated on-site childcare facility**
- **Children under 15** are **not permitted on the show floor** and must be accompanied to the childcare facility upon arrival

Booking Childcare

To arrange childcare, email the Registration Team at registration@londontechweek.com — they will set up the service and guide you through the required **Child Attendance Waiver**.

⌚ Claude synthesis (Tier 2) · distance 0.759

London Tech Week 2026 Assistant

8–12 June 2026 · Olympia London

Ask about London Tech Week...



7. Challenges, Debugging, and Learnings

Several technical problems came up across the project. Working through them was where much of the real learning happened.

- **Python version incompatibility.** The Anthropic library's import hung indefinitely on Python 3.14. I diagnosed this by isolating the import from the network call to see which step was actually freezing, and fixed it by rebuilding the virtual environment on the stable Python 3.12. Lesson: for a deadline project, the newest version is not always the best choice.
- **Retired model name.** An out-of-date model string (claude-sonnet-4-5) caused an error; correcting it to claude-sonnet-4-6 fixed it. Lesson: model names are retired over time, so it is worth checking the current documentation.
- **Streamlit import hang.** Streamlit failed to import due to a protobuf version mismatch — the installed protobuf 6.x was incompatible with the Streamlit version, which needed a 4.x–5.x release. Downgrading protobuf resolved it. Separately, I found that VS Code's integrated terminal caused commands to freeze, so I switched to running commands in the macOS Terminal app and used VS Code only for editing.
- **A cloud-sync gremlin.** Because the project lived in an iCloud-synced folder, some installed library files were stored only in the cloud rather than locally, which caused imports to stall while the system tried to download them. Forcing the project folder to download and stay downloaded resolved it.
- **Git hygiene.** I checked git status before every push to make sure the .env file (which holds the API key) was never committed, and confirmed that the regenerable chroma_db/ folder was correctly gitignored.

Reflections on Key Lessons Learned

In addition to addressing specific issues, I identified several broader lessons. First, maintaining rigorous grounding by ensuring every answer is supported by verified source data distinguishes a reliable assistant from one that appears confident but may generate inaccuracies. I observed this when Visual Studio Code's AI suggested a Q&A without verifying the source. Second, effective systems are developed incrementally through small, testable steps. Most challenges were resolved by isolating individual components rather than relying on conjecture. Third, making thoughtful engineering trade-offs, such as selecting Sonnet instead of Opus, using a local embedding model, and confirming server-side rendering before employing Selenium, helps keep projects cost-effective and manageable within time constraints.

8. Limitations and Future Improvements

The project is functional and meets the brief, but there are clear, honest opportunities to extend it:

- **Link speakers to sessions.** Sessions and speakers are currently stored separately, so “who is speaking at session X?” cannot always resolve to a named person. Linking the two datasets would close this gap.
- **Document-level fallback.** The current Tier 2 fallback synthesises answers from the closest Q/A pairs rather than from the full raw scraped documents. Synthesising from curated Q/A pairs keeps answers grounded in checked content, but adding a separate vector collection over the raw documents — as a third retrieval layer — would let the bot answer questions that fall outside the Q/A set. This was a deliberate scoping decision made to keep a working system stable before the deadline.
- **Retrieval tuning.** Building a small evaluation set of test questions would allow the threshold and the number of retrieved results to be tuned systematically, improving cases where the right context is not currently retrieved into the top 3.
- **Live web-search fallback (Tavily).** Adding a live web-search tier would let the assistant answer questions that the London Tech Week data does not cover at all.
- **Voice interface (ElevenLabs).** Adding voice output would improve accessibility and make for a strong demonstration.

9. Architecture Summary

The end-to-end pipeline flows as follows:

```

Scrape (requests + BeautifulSoup + lxml, via XML sitemaps)
|
v
Raw data (sessions.json, speakers.json)
|
v
Q/A generation (Claude claude-sonnet-4-6, structured JSON) + 22 hand-verified pairs
|
v
qa_pairs.csv (172 pairs: source, question, answer)
|
v
Embed + store (ChromaDB, all-MiniLM-L6-v2, local) -> chroma_db/
|
v
Hybrid retriever (Tier 1 direct hit OR Tier 2 Claude RAG fallback)
|
v
Streamlit UI (chat, memory, source citation, clear chat)

```

A diagram of this pipeline is included in the repository README. The repository follows the recommended structure, with `scraper.py`, `qa_generator.py`, `vector_store.py`, `retriever.py`, and `app.py` separated by responsibility, plus the `data` folder, `requirements.txt`, and this report.

10. Conclusion

Building on these lessons, this project implements a complete RAG pipeline for London Tech Week 2026: from scraping a real website via its sitemaps, through generating and hand-curating a 172-pair Q/A dataset, to storing it in a local vector database and serving it through a hybrid two-tier retriever wrapped in a branded Streamlit chat interface. Throughout the project, I consistently worked through real-life debugging challenges and made deliberate engineering trade-offs, keeping the system both affordable and reliable. The end result is a working assistant that answers practical questions about a major in-person event and a much clearer understanding of how retrieval-augmented systems are built end-to-end.